



Invest-Tracker – Behavioral Interview

Prep

All answers follow the **STAR format** (Situation → Task → Action → Result).

These are grounded in your **actual code** – not made up stories.

1. "Tell me about a challenge you faced and how you overcame it."

Topic: CORS errors during deployment

Situation:

When I deployed the frontend to Vercel and the backend to a separate Vercel serverless deployment, the app worked fine locally but completely broke in production. Every API call from the frontend was being blocked.

Task:

I had to figure out why the backend was rejecting requests from the frontend even though I had CORS configured.

Action:

I investigated and found two issues. First, my CORS config only allowed `localhost` origins – I hadn't added the production Vercel URL. Second, preflight `OPTIONS` requests were failing because they weren't going through the same CORS middleware. I fixed this by:

- Maintaining an explicit `allowedOrigins` array in `server.js` with both localhost and the Vercel domains
- Using `app.options('*', cors(corsOptions))` to handle preflight requests with the same config
- Adding a `FRONTEND_URL` environment variable so I could add new origins without changing code

Result:

The app started working in production. I also learned to always test with production URLs before considering deployment complete.

2. "Tell me about a time you had to learn something new quickly."

Topic: Supabase Auth & JWT tokens

Situation:

I chose Supabase for authentication because it handled user management out of the box. But I quickly realized I didn't understand how the auth token flow worked between my React frontend and Express backend.

Task:

I had to make the backend trust and validate the user's identity on every private API request.

Action:

I read through the Supabase docs and understood that after login, Supabase gives a JWT `access_token`. I then:

- In the frontend's `api.js`, I wrote an Axios **request interceptor** that automatically calls `supabase.auth.getSession()` before every API call and attaches the token as `Authorization: Bearer <token>` in the header.
- In the backend's `auth.js` middleware, I call `supabase.auth.getUser(token)` to verify the token server-side with Supabase, then attach the verified user to `req.user` for use in route handlers.

Result:

Every protected route like "get my stocks" or "add a stock" now automatically verifies who the user is. I didn't need to build a separate login system or store passwords myself.

3. "Describe a time you made a mistake and what you learned from it."

Topic: Exposing the Finnhub API key on the frontend

Situation:

In the early stages, I was calling the Finnhub stock price API directly from the React frontend to keep things simple.

Task:

I needed to display live stock prices, and calling from the frontend seemed the quickest approach.

Action:

I realized that having the API key in the frontend code means anyone could open browser DevTools, see the key in the Network tab, and abuse it — which could lead to my account being banned or charged. So I moved all Finnhub API calls to the **Express backend**. The frontend now calls my own backend (`/api/stocks/search`), and the backend secretly calls Finnhub with the API key stored in a `.env` file that never reaches the client.

Result:

The API key is now completely hidden from users. This also gave me a single place to manage rate limiting and error handling for external API calls.

4. "Give an example of how you handled a technical problem under pressure."

Topic: Real-time stock prices causing API rate limit errors**Situation:**

After adding more stocks to the portfolio, the app started throwing errors. Finnhub has a free-tier rate limit, and the `updateStockPrices` function was calling the API for every single stock simultaneously.

Task:

Fix the price update logic without hitting rate limits and without breaking the user experience.

Action:

I looked at the `stockPriceService.js` code and added a **1-second delay** between each Finnhub API call using `await new Promise(r => setTimeout(r, 1000))` inside the for-loop. This ensures stocks are updated sequentially, not all at once. I also added a 7-second timeout on each Axios call so a slow response doesn't hang the entire server.

Result:

Rate limit errors disappeared. The price updates now run every 5 minutes in the background in a controlled, sequential manner.

5. "Tell me about a project you are proud of and why."

Topic: The overall Invest-Tracker project**Situation:**

I wanted to build something I would actually use — a personal finance app to track my stock holdings,

see live P&L, and manage other assets like gold.

Task:

Design and build a full-stack web application end-to-end, from database design to deployment.

Action:

I built a complete full-stack app with:

- **React + Vite** for the frontend with Tailwind for styling and Recharts for portfolio graphs
- **Express.js** backend with clean REST APIs
- **Supabase** for the database (PostgreSQL) and authentication
- **Finnhub API** for live stock prices
- **Deployed both frontend and backend on Vercel**

I also implemented useful features like:

- Protected routes so only logged-in users see their data
- A stock search feature that filters results to only show US exchange stocks (NASDAQ, NYSE, etc.)
- Portfolio summary calculation (total value, total gain/loss %) on the backend

Result:

A fully working, deployed application. I can actually log in and track my own investments with it today.

6. "Describe a situation where you had to make a decision with incomplete information."

Topic: Choosing Supabase vs building auth from scratch

Situation:

When planning the project, I had to decide how to handle user authentication. I could build it myself using JWT + bcrypt, or use a managed service like Supabase.

Task:

Make a decision quickly without having experience with either approach in a real project.

Action:

I researched both options. Building from scratch gives full control but adds complexity — I'd have to handle password hashing, token refresh, email verification, and security vulnerabilities myself. Supabase handles all of this and also gives a PostgreSQL database. Since my goal was a working product and learning full-stack development — not building auth infrastructure — I chose Supabase.

Result:

I shipped the auth feature much faster and focused my time on the actual features of the app. The

tradeoff was some vendor dependency, which I was comfortable with for this project.

7. "Tell me about a time you improved something."

Topic: Stock search result quality

Situation:

The stock search feature was returning too many irrelevant results. When a user typed "Apple", they'd see results from foreign exchanges, ETFs, and derivative instruments alongside the actual Apple stock (AAPL).

Task:

Improve the quality of search results without a major refactor.

Action:

In the `/api/stocks/search` route, I added a tiered filtering strategy:

1. **First preference:** Show only stocks from major US exchanges — NASDAQ, NYSE, BATS, ARCA
2. **Fallback:** If no exchange matches, show stocks with type "Common Stock"
3. **Last fallback:** Show anything with a symbol and description

I also limited results to 10 items using `.slice(0, 10)`.

Result:

Users now see relevant, clean results when searching for a stock. The search is much more intuitive.

8. "How do you handle situations when things don't go as planned?"

Topic: Supabase auth token not reaching the backend

Situation:

After building both the login system and the backend API, clicking any protected action like "Add Stock" gave a `401 Unauthorized` error. The auth token wasn't making it to the backend.

Task:

Debug why the token wasn't being sent, even though the user was clearly logged in.

Action:

I added detailed `console.log` statements at every step — in the frontend Axios interceptor to confirm the token was being read, and in the backend auth middleware to confirm it was being received.

This is actually visible in the code today (the 🔍 `Auth Middleware Debug` and 🔍 `API Request Debug` logs). Through this, I found the real issue: the Axios instance wasn't calling `supabase.auth.getSession()` asynchronously correctly – it was resolving before the session was loaded on first page render. I fixed it by making the interceptor `async` and `await`-ing the session call properly.

Result:

Auth started working end-to-end. I kept the debug logs in place as a useful monitoring tool during development. This experience taught me the importance of logging at every boundary between systems.

Quick Cheat Sheet – Common Follow-up Answers

Question	Quick Answer
Tech stack?	React + Vite frontend, Express.js backend, Supabase (PostgreSQL + Auth), Finnhub API, deployed on Vercel
Why Supabase?	Handles auth + database together, saves time, no need to manage passwords or JWTs manually
Why separate frontend/backend?	Separation of concerns – backend keeps API keys hidden, can scale independently
Biggest learning?	Never expose API keys on the frontend. Always call 3rd party APIs from the server
What would you do differently?	Add loading states earlier, write tests, and use environment-based config from day one
How did you test?	Manually tested each API route (used the <code>/test-finnhub/:ticker</code> debug endpoint I built), tested frontend flows in browser